

Guía de orientación — Taller 3 (Parte 2): llevar nuestra historia a Enterprise SDD

Guía de apoyo, no un examen ni una imposición. Es el mapa para que cada equipo retome lo que ya hizo en el Taller 2 y lo **implemente** pasándolo por el framework Enterprise SDD del libro de Papalini.

0. Por qué hacemos esto (leer primero)

En el Taller 2 construimos la base: constitución, spec, plan y test-cases. Ahora toca **implementar** esa misma historia, pero llevándola al nivel de **Enterprise SDD**.

Seamos claros con el objetivo, porque define cómo trabajamos:

- **No es** adoptar Enterprise SDD como nuestro framework definitivo.
- **Sí es** usarlo como un *laboratorio*: ejecutarlo con nuestra propia historia para entender **por qué el autor decide lo que decide**, comparar contra OpenSpec/SpecKit que ya conocemos, y de ahí **extraer los patrones que nos convenzan**.
- La meta final es sentarnos, ya con criterio, a definir **nuestro propio marco** (Lumería SDD / Vonex SDD). Lo que cada equipo aprenda aquí es insumo de esa decisión.

Como decía Rony en la sustentación: a un framework nuevo hay que tomarlo "con pinzas". De acuerdo. Por eso lo estudiamos por sus **patrones** (que no caducan), no por su versión. El propio autor lo dice: *"el andamiaje analítico —las doce dimensiones, los diecinueve patrones— debería seguir siendo útil sin importar las versiones"* (cap. 9.5 / introducción).

Lectura base (sin examen): Parte 1 y Parte 4 del libro. La Parte 4 son los capítulos 11, 12, 13 y 14 — justamente Enterprise SDD. Al final de esta guía hay un mapa de qué sección leer para cada tema.

1. Cómo arranca tu equipo según tu proyecto

Casi todas nuestras historias se montan sobre un **proyecto que ya está en producción** (Lumeria, Vonex y los demás), trabajado con Scrum. Eso define el camino de instalación. El libro plantea dos modelos (cap. 11.6, Apéndice A):

Tu caso	Modelo	Cómo se instala
Historia sobre un proyecto que ya corre (brownfield: Lumeria, Vonex, etc.)	Modelo B (recomendado para repos existentes)	Agregas el framework como submódulo (tools/enterprise-sdd) y sincronizas sus carpetas (.github/, .specify/, .sdd-modules/...) al raíz de tu repo. No tocas la rama de producción: trabajas en una rama/feature aparte.

Tu caso	Modelo	Cómo se instala
Historia nueva o repo limpio (greenfield)	Modelo A (arranque rápido)	Clonas el framework como base del proyecto y construyes encima.

Los pasos exactos de cada modelo están en `INSTALL - IN - NEW - PROJECT .md` y `TEAM - ADOPTION - GUIDE .md` del propio repo. **Brownfield con documentación previa** (PRDs, ADRs, specs ya escritas): el comando `sdd ingest ./docs/` las clasifica y mapea a la estructura SDD para acelerar el arranque.

Regla de oro para los que trabajan sobre producción: Enterprise SDD se monta *encima* de tu código, no lo reescribe. Tu rama de feature y tus gates son tu zona de trabajo; `main`/producción no se toca hasta el final.

Reutiliza lo del Taller 2. No empieces de cero. Tu `constitution.md`, `spec.md`, `plan.md` y `test-cases.md` ya son los insumos de las Fases 0 a 3. Esta semana se trata de **cerrar los huecos que señalamos y codear**.

2. El recorrido: 6 fases, 4 gates (la columna vertebral)

Enterprise SDD instancia el ciclo SDD en seis fases con cuatro compuertas (cap. 11.4). Esta es toda la estructura que necesitas tener en la cabeza:

Fase	Qué produce	Gate al cerrar
0 • Constitución	<code>constitution.md</code> (8 artículos)	— (prerrequisito de todo)
1 • Especificación	<code>spec.md</code> (US-XXX, AC-XXX, NFR-XXX)	Gate 1 — spec completa, sin [NEEDS_CLARIFICATION]
2 • Diseño	<code>plan.md</code> + contratos (OpenAPI/eventos)	Gate 2 — cada US mapea a un componente
3 • Preparación (tests + tareas)	<code>test-cases.md</code> (TC-XXX) + <code>tasks.md</code> (T-XXX)	Gate 3 — cada AC cubierto por un TC; análisis $\geq$ PASS
4 • Implementación	código en <code>src/</code> + tests (TDD rojo $\rightarrow$ verde)	—
5 • Revisión & Ship	<code>ship-checklist.md</code> + revisión de seguridad	Gate 4 — re-valida 1, 2 y 3; sin TODO/FIXME

Dos ideas que valen oro:

- **Gate 4 vuelve a correr los gates 1, 2 y 3** sobre el estado actual de todo (cap. 11.5). Es el que atrapa el *drift* entre spec y código antes de que llegue a producción.
- El "**modelo de la semana de Marta**" (cap. 11.1): un día por fase. Lunes constitución + spec, martes diseño, miércoles tests + tareas, jueves implementación, viernes ship. No

escribió una sola línea de código de producción hasta el jueves — los tres días previos fueron el andamiaje que hizo que esos dos últimos volaran. Ese es el ritmo a imitar.

Quién interviene en cada fase: los agentes

El libro cataloga los agentes por fase (Tabla 12.1 y cap. 12.5). **No los usas todos en cada historia:** hay un **núcleo de ~10** que recorre el camino completo, y **opcionales** que se activan según lo que tu historia tenga (API, eventos, BDD). Arranca con el núcleo.

Fase	Agente	Núcleo	Qué hace
pre-1	@brainstorming	opcional	Explora el espacio de solución (12 perspectivas); solo resumen en chat, sin artefacto.
0	@constitution		Crea <code>constitution.md</code> . Corre una vez por proyecto (o por enmienda).
1	@requirement-analyst		Único que escribe historias de usuario. Modo Vision → <code>business-context.md</code> ; modo Detailed → <code>spec.md</code> (US/AC/NFR en Dado-Cuando-Entonces).
1	@clarification		Aflora supuestos ocultos y enruta cada duda al rol correcto (PO / Dev Lead / QA). Produce <code>clarifications.md</code> .
2	@architect		El agente más potente. Diseña la arquitectura, clasifica el cambio como NEW/EXTEND/HYBRID, usa diagramas Mermaid. Nunca escribe código. Produce <code>plan.md</code> y <code>data-model.md</code> .
2	@api-champion	opcional	Contratos OpenAPI 3.0+ (naming, versionado, errores, paginación). Si expones

Fase	Agente	Núcleo	Qué hace
			API REST.
2	@messaging-champion	opcional	Contratos AsyncAPI 3.0+ (eventos, colas, dead-letter). Si hay mensajería/colas.
3	@test-explorer		Estrategia y casos TC - XXX desde la spec; matriz AC → TC. Cubre happy path, errores, autorización, borde, concurrencia y NFR.
3	@gherkin-analyst	opcional	Archivos <code>.feature</code> en BDD, etiquetando cada escenario con <code>@US-XXX/@AC-XXX</code> . Si hacen BDD.
3	@software-engineer (Planning)		Convierte plan + spec + tests en <code>tasks.md</code> (tareas T-XXX de 2-8 h con dependencias).
3	@analysis		Guardián del Gate 3: verifica la trazabilidad completa, halla huérfanos y contradicciones. Veredicto PASS / PASS-WITH-WARNINGS / FAIL.
4	@test-engineer		TDD Red: convierte los test-cases en código de prueba que debe fallar primero . Nunca toca <code>src/</code> .
4	@software-engineer (Implementation)		Escribe el mínimo código para pasar los tests, una tarea a la vez. Protocolo del 70%: 3 intentos fallidos → para y escala.
5	@review		Produce el <code>ship-checklist.md</code> del Gate 4 (cumplimiento, calidad, cobertura, docs, performance). Veredicto APPROVED ... DO NOT SHIP.

Fase	Agente	Núcleo	Qué hace
5	@security-reviewer		Evidencia de seguridad del Gate 4: OWASP Top 10 + secretos + cadena de suministro. Un hallazgo CRITICAL bloquea el gate.
5b	@refactoring	opcional	Analista read-only; propone <code>refactoring-plan.md</code> (deuda técnica, violaciones), sin escribir el código.
mant.	@tech-context-maintainer	opcional	Detecta <i>drift</i> entre código, spec y constitución; produce reportes de desviación.

@software-engineer es **un solo agente con dos modos** (Planning en Fase 3, Implementation en Fase 4); por eso el núcleo son ~10 agentes distintos. Existen además 5 **meta-agentes** (@agent-builder, etc.) que sirven para construir el framework, **no** para tu feature.

3. De git clone a tu primer feature (pasos concretos)

Lo que necesitas antes: Python ≥ 3.11, Git, Bash, tu IDE (VS Code o **Claude Code**), y un `.env` con tu token (parte de `.env.example`). Verifica con `sdd doctor`.

Entiende el patrón antes de teclear (y léelo en el libro, que para eso está): en cada fase el ciclo se repite — `sdd` te crea la ranura vacía del artefacto → **tú invocas en el chat al agente de esa fase** (los de la tabla de arriba) para llenarla o refinarla → corres su gate. Cómo se ve esto de principio a fin lo cuentan la *semana de Marta* (cap. 11.1) y el *recorrido completo de comandos* (cap. 12.13). Esta guía es el mapa; el detalle, invéstiguenlo ahí.

1. Clona el framework: `https://github.com/popoloni/enterprise-sdd` e instálalo según tu modelo (A o B del punto 1; comandos exactos en `INSTALL-IN-NEW-PROJECT.md`). El CLI se instala con `pip install -e .specify/cli`.
2. **Genera los agentes para Claude:** `sdd adapters generate --target claude`. El framework trae 16 agentes (en tu historia usarás ~10; ver la tabla de arriba) y los *adapta* a tu herramienta (soporta Claude Code, Copilot, Cursor, Codex). Así invocas @constitution, @architect, etc. desde Claude. Verifica con `sdd doctor`.
3. Crea el feature: `sdd new <id-feature> --level standard` (`--level` fija la ceremonia: `ultra-light` / `standard` / `full`). (En `INSTALL-IN-NEW-`

PROJECT.md los ejemplos usan `--template standard`; resuelve a lo mismo — el flag de ceremonia es `--level`.)

4. **Porta tus artefactos del Taller 2** a las ranuras que crea el framework: `constitution.md` va en `.specify/memory/`; `spec`, `plan` y `test-cases` bajo `.specify/specs/<id-feature>/`. Peguen lo que ya tienen y **usen los agentes para cerrar los huecos que señalamos y avanzar a lo que aún no hicimos** (tests ejecutables y código), no para regenerar desde cero lo que ya está bien. Pista para investigar: `@architect` clasifica el cambio como NEW/EXTEND/HYBRID — para un proyecto que ya corre, averigüen qué implica esa clasificación (cap. 12.5).
5. Corre los gates en orden: `sdd gate <id> 1`, luego 2, luego 3. Si uno sale **rojo**, te dice exactamente qué falta: lo arreglas y vuelves a correr. Un gate rojo no es un fracaso, es el framework haciendo su trabajo (cap. 11.5).
6. Implementa la Fase 4 con TDD: test que falla → código mínimo → verde, un commit por tarea. Apóyate en `sdd analyze --gaps` para ver código sin respaldo en la spec.
7. Cierra con `sdd ship` (Fase 5) y apunta a Gate 4 en verde. (Existe además un `sdd gate post-merge` opcional para validar después del merge.)

4. Cómo encaja con nuestro Scrum (los Three Amigos)

No hay choque entre Scrum y Enterprise SDD. El framework mapea los roles que ya tenemos (cap. 12.8): *"Los agentes amplifican cada rol; no lo reemplazan."*

Rol Scrum	Fase que lidera	Artefacto que firma
Product / PO	Fase 1 (Spec)	<code>spec.md</code> → firma Gate 1
Tech Lead	Fase 2 (Diseño)	<code>plan.md</code> → firma Gate 2
QA	Fase 3 (Preparación)	<code>test-cases.md</code> → firma Gate 3
Todos	Fase 5	<code>ship-checklist.md</code> → Gate 4

Si su tablero es Jira/GitHub Issues, el framework puede sincronizar `tasks.md` con el tracker (`sdd sync`, cap. 13.8): cada T-XXX se vuelve un issue del sprint, sin pelear por cuál es la fuente de verdad. Es opcional; si su historia es pequeña, el `tasks.md` local basta.

5. Cuánta ceremonia usar: NO sobre-ingenierizar

Este es el error más fácil de cometer con un framework de 16 agentes. El autor es explícito (cap. 10.7): *"El proceso correcto es la cantidad más pequeña que previene tus fallos, más una sola capa de margen. Cualquier cosa más es overhead que el equipo terminará rechazando."*

Hay tres niveles de ceremonia (cap. 12.10). **Arrancamos en Standard:**

- **Ultra-light** — para cambios triviales (un typo, una config de una línea). Salta Fases 2-3.
- **Standard** — *nuestro punto de partida*. Las seis fases, los cuatro gates, profundidad normal.
- **Full governance** — solo para lo de alto riesgo: auth, pagos, datos sensibles (PII).

No instalen "todo el framework" porque se ve completo. Empiecen con lo Standard y suban ceremonia **solo** cuando un fallo concreto lo justifique. Menos es más.

6. Autonomía: por ahora, humano en el loop

Enterprise SDD permite ejecución autónoma con presupuesto de llamadas y protocolo de escalación (cap. 14). **No es por donde arrancamos**. Trabajamos en el **modo de ejecución standard** (humano conduce cada fase) porque estamos aprendiendo el framework.

Ojo con el nombre: este standard es el *modo de ejecución* (qué tan autónomo es el agente) y es un **eje distinto** del *nivel de ceremonia standard* de la Sección 5 (cuánto proceso). Coinciden en el nombre, no en el significado — el libro lo aclara en "*Ceremony ≠ autonomy*" (cap. 12.10). Vale la pena que confirmen esa diferencia leyéndola.

Conozcan los conceptos —budget, one-item-per-cycle, escalación P1/P2/P3— pero la autonomía la probaremos después, cuando ya tengamos specs y tests maduros. Como dice el libro, la autonomía **no es la meta**; la disciplina de comportamiento va primero (cap. 8.1.4).

7. Errores que el libro nos pide evitar (cap. 10.4)

De los ocho anti-patrones, estos son los que más nos rondan:

- **Saltarse fases por deadline** → los gates son programáticos justamente para que no se puedan saltar "porque hay apuro".
 - **Trazabilidad de adorno** → corran `sdd analyze --gaps` antes de merge; no es decorativo.
 - **Tests escritos después del código** → la Fase 3 produce los test-cases *antes* de codear.
 - **Spec que nadie lee** → resumen ejecutivo de una página al inicio de cada artefacto.
 - **Agente que te da la razón sin verificar** (sicofancia) → pídanle que cuestione, no que adule.
-

8. El espíritu del taller (criterio, no obediencia)

Lean al autor con la guardia arriba. Que **les tenga que convencer**; si un patrón no los convence, está bien descartarlo. El libro mismo trae una "rejilla de adopción" (cap. 10.2: ADOPT / TRIAL / ASSESS / HOLD) — úsenla como lente. Lo que sí está probado y vale adoptar ya: spec antes de código, plantillas con secciones obligatorias, constitución, gates programáticos. Lo demás, se

evalúa.

Y recuerden la tesis central (cap. 10.3): los frameworks **no compiten, se complementan**. La pregunta no es "¿qué framework adopto?", sino "¿qué combinación cubre lo que necesito?".

9. Qué entregamos al cierre de la Parte 2

1. **Repo con la historia implementada** vía Enterprise SDD (rama de feature, no producción).
2. **Artefactos portados** y gates en verde (evidencia: salidas de `sdd gate` y el `ship-checklist.md` del Gate 4).
3. **Bitácora de implementación** (el `cycle-log` que el framework va dejando).
4. **Media página de reflexión por equipo**: ¿qué 2-3 patrones de Enterprise SDD nos convencieron y cuáles no, y por qué? Esto es lo más importante: es el material con el que, juntos, vamos a armar nuestro marco propio.

Bonus: para esta fase habilitamos Claude del equipo en las máquinas con acceso.

Mapa de lectura del libro (para cuando se atasquen)

Tema	Dónde leer
Visión general y la semana de Marta	Cap. 11.1
Las 3 filosofías (constitución, trazabilidad, gates)	Cap. 11.3
Las 6 fases en detalle	Cap. 11.4
Cómo funcionan los 4 gates	Cap. 11.5 / 12.9
Constitución en la práctica (8 artículos)	Cap. 11.6
Instalar de cero (Modelo A y B)	Cap. 11.6 + Apéndice A
Catálogo de los 16 agentes	Cap. 12.5
Roles humanos ↔ fases (Three Amigos)	Cap. 12.8
Niveles de ceremonia	Cap. 12.10
Un recorrido completo de comandos	Cap. 12.13
Sincronizar con Jira/Issues	Cap. 13.8
Módulos y presets (Java/React, etc.)	Cap. 13.5–13.6
Ejecución autónoma, budget, escalación	Cap. 14.2, 14.4, 14.5, 14.7
Qué adoptar / qué evitar	Cap. 10.2
Ruta de migración por etapas	Cap. 10.5
La trampa del "más es más"	Cap. 10.7

